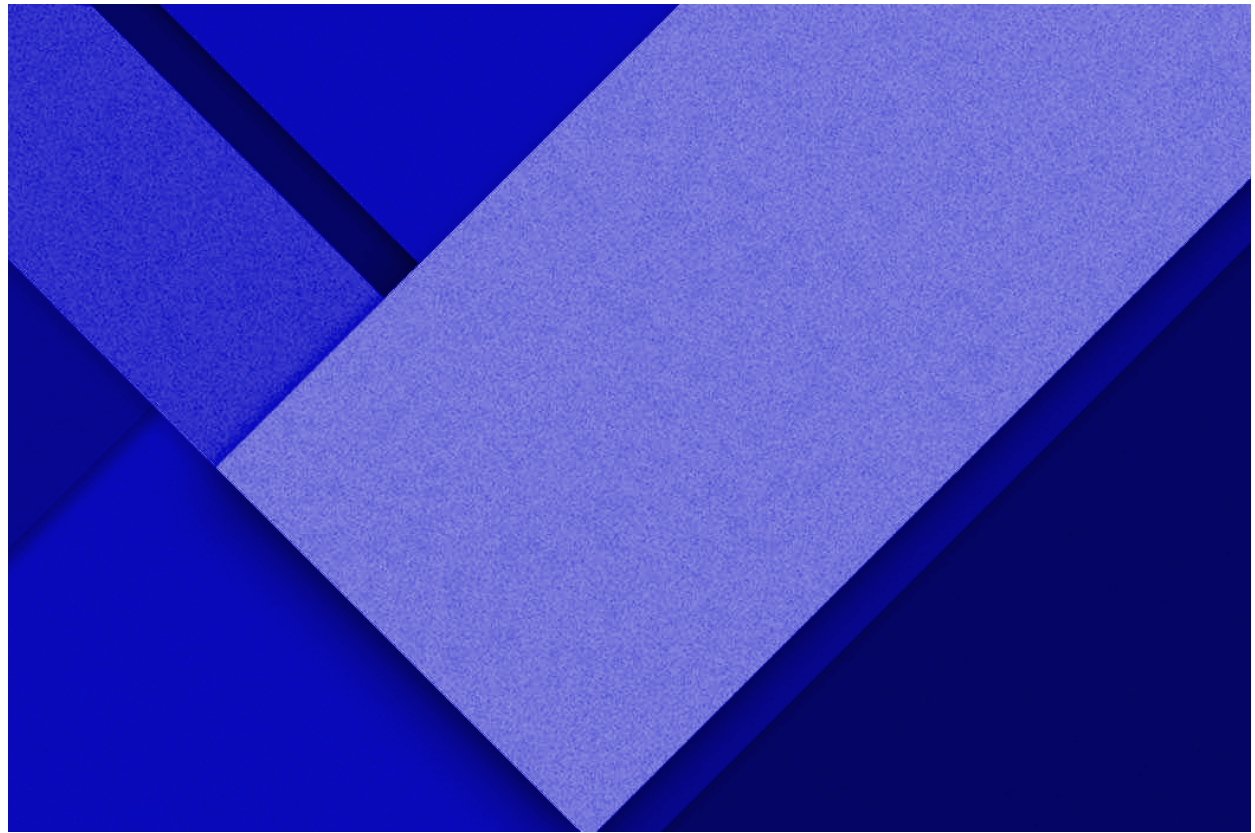




OpenAni

Alejandro Stoian

Stephano Rayden Leon Estela

Fabio Junior Ninahuanca



1. Introducción.....	3
2. Estudio de mercado.....	4
2.1.	4
2.2. DAFO.....	4
2.2.1. Fortalezas.....	4
2.2.2. Oportunidades.....	4
2.3. Cliente.....	5
3. Arquitectura de la aplicación.....	6
3.1. Estructura BB.DD.....	6
3.1.1. Entidades.....	6
3.1.2. Relaciones.....	6
3.2. Android Studio y React Native.....	7
3.2.1. Historias de usuario.....	8
3.2.2. Definición de componentes.....	8
3.3. Chat en directo.....	8
3.3.1. Backend.....	9
3.3.2. Frontend.....	9
4. Estilo de la aplicación.....	10
4.1. Paleta de colores.....	10
4.2. Fuente.....	10
4.3. Elementos visuales.....	10
5. Memoria del desarrollo.....	12
5.1. Fase de diseño.....	12
5.2. Backend.....	16
5.3. Frontend.....	28

1. Introducción

Cada día es mayor el problema que suponen las plataformas de streaming de pago en todo el mundo. En 2025, Netflix aumentó el coste de su suscripción una cantidad considerable, enmascarándola como un “ahorro del 30 %”¹ dando como razón una mejora de servicios.

No fue la única empresa en recurrir a la subida del precio de sus servicios y, sumándose a esto la gran cantidad de plataformas de streaming existentes hoy en día, hace que sea cada vez más difícil para el consumidor mantener todas las suscripciones activas.

El anime se ve afectado por la situación también. La serie “JoJo's Bizarre Adventure” se volvió propiedad de Netflix en 2021, convirtiéndola en la única plataforma en la que se puede ver. Nuestra aplicación ofrece un lugar donde poder encontrar todos los animes que uno se puede imaginar, incluso aquellos que no se encuentran en las plataformas mainstream.

El principal enfoque de la aplicación es funcionar como portal para poder descargar torrents de animes y mangas a través de la API de nyaa.si, una página web con funcionalidad parecida pero sin una versión móvil y con una interfaz primitiva. Todo esto sin tener que preocuparse ni de pérdida de calidad en la imagen, ni de suscripciones.

¹ [Netflix sube el precio de sus tarifas en España a partir de septiembre: este es el dinero que pagarás de más](#) (Bussiness Insider - 04 Ago 2025)

2. Estudio de mercado

2.1.

2.2. DAFO

2.2.1. Fortalezas

- Código libre: Todo es transparente, el funcionamiento interno de la aplicación es visible y modificable para que se pueda mejorar y personalizar.
- Sistema de chat y comentarios: Poder enviar mensajes y poner comentarios permite a los usuarios dejar feedback y comunicarse entre ellos sobre los torrents.
- API duradera: Nyaa Torrents lleva activa desde el 2005, lo cual la convierte en un sistema API de confianza, sin gran probabilidad de caer.
- Sistema de torrents: La propia página de [Nyaa.si](https://nyaa.si) no permite la subida de archivos de calidad baja, manipulados o con codecs cambiados. Todo el contenido que se sube a la API está verificada y moderada muy rigurosamente. Esto implica que los usuarios puedan ver anime en, muchas veces, calidad incluso más alta que la encontrada en plataformas de streaming.

2.2.2. Oportunidades

- Subida de precios: Mencionado en la introducción, la subida de los precios constante de las plataformas de streaming atrae a nuevos usuarios activamente.
- Eventos y convenciones: Nuestra aplicación podría salir promocionada en convenciones locales, ya sea de boca a boca o publicidad directa.

2.2.3. Debilidades

- Incapacidad de visualizar anime directamente en la aplicación: Muchos clientes potenciales se verán repelidos por no poder consumir anime o manga sin previamente descargarlo.
- Necesidad de una aplicación externa para la descarga de torrents

- Almacenamiento móvil: Como los torrents se descargan localmente, el cliente necesita disponer del almacenamiento necesario.

2.2.4. Amenazas:

- Páginas web que permiten ver anime sin necesidad de descarga.
- Legalidad y Distribución: Google Play y App Store suelen banear aplicaciones que facilitan la descarga de contenido con copyright, por lo que tendríamos que apoyarnos en otros medios para distribuir nuestra aplicación, como GitHub directamente o FDroid.

2.3. Cliente

Esta aplicación está disponible para personas de todo el mundo y está pensada para quienes estén cansados de pagar múltiples suscripciones solo para poder ver una serie, leer, o consumir anime y manga.

Está dirigida a todo público sin una edad específica, ofreciendo una experiencia libre y centrada en los gustos de los amantes de la cultura asiática. Aunque la edad no influye en nuestros usuarios, lo que sí importan son los hobbies. Los usuarios de nuestra aplicación serían principalmente consumidores de este tipo de contenido. Por culpa de usar un sistema de torrents, los usuarios tendrán, en su mayoría, conocimientos tecnológicos básicos.

Al ser una aplicación móvil, está dirigida a aquellas personas que disponen de un dispositivo móvil inteligente, que sea capaz de manejar torrents.

3. Arquitectura de la aplicación

3.1. Estructura BB.DD.

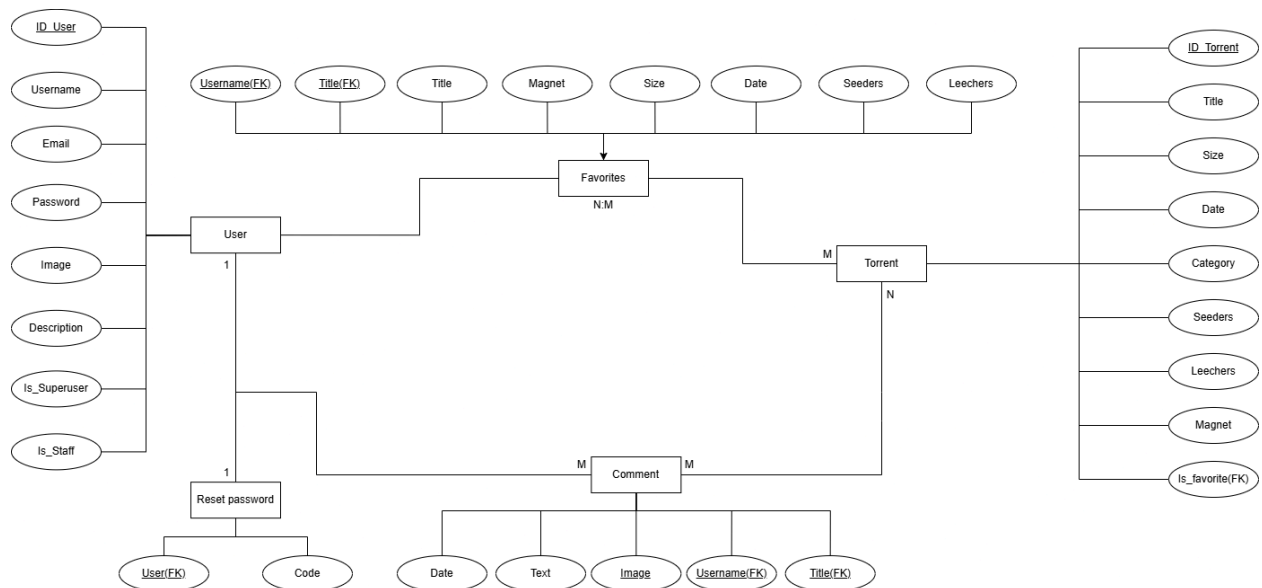
3.1.1. Entidades

- **User:** Almacena los datos de registro y perfil. Cada usuario puede tener muchos torrents como favoritos, y este mismo puede escribir muchos comentarios. El ID único es Key (ID_User, Username, Email, Password, Description, Image)
- **Torrent:** Desde la API se rescatan datos y se añaden a la base de datos, cada torrent tiene muchos comentarios y muchos usuarios pueden guardarlo como favorito. El ID único es Key (ID_Torrent, Title, Size, Date, Category, Seeders, Leechers, Likes, Dislikes, Enlace)
- **Comment:** Cada usuario puede hacer uno o muchos comentarios, y en esta entidad se guardan datos, algunos de los datos son recogidos por acciones de usuarios, como el escribir, el dar like o dislike, y automáticamente se genera la fecha en la que se publicó el comentario. El ID único es Key (ID_Comment, Text, Like, Dislike, Date).
- **Room:** Esta entidad sirve para la conexión entre dos usuarios para que se puedan comunicar mediante mensajes en tiempo real. (user1, user2, updated)
- **Message:** Un usuario puede enviar un mensaje a un usuario. Una vez se vaya a enviar el mensaje, se crea un room, si no está creado antes, para que estos se puedan comunicar en tiempo real. Cada mensaje enviado guarda: El room en el que se conectará el usuario, el remitente del mensaje, el contenido del mensaje y la fecha en la que se envió. (room, sender, text, timestamp)

3.1.2. Relaciones

- **Favorites:** Para relacionar las tablas, hemos creado otra tabla en la que introducimos el título de cada torrent seleccionado por cada usuario, entonces cuando el usuario marque algún torrent como favorito, se agregan los datos del torrent a esta tabla.
- **Reset Password:** Genera un código de uso único para permitirle al usuario cambiar su contraseña. El usuario debe introducir su correo y generar un código utilizando un botón.

3.2. Android Studio y React Native



Usando Java, la funcionalidad se gestiona con la utilización de varias librerías y dependencias. Para el manejo de la base de datos creada en Django, se necesita trabajar con JSONs. Para ello se usa Retrofit con el gestor Gson. Además se hará uso de Glide, elementos como RecyclerView e Intent implícitos para mandar el enlace .torrent a una aplicación externa (BitTorrent®, uTorrent®, Flud®, etc.)

3.2.1. Historias de usuario

Se hacen peticiones constantes a la API, nuestra aplicación está constantemente en línea. Las peticiones posibles a la API de Nyaa son para recibir todos los animes en la pantalla principal. Esta después se podrá filtrar por categorías, indicadas en un menú lateral en la aplicación. Por una barra de búsqueda se podrá filtrar por nombre.

De la base de datos servidor Django que montaremos se harán peticiones para mostrar información de la cuenta de usuario y los comentarios. Los torrents solo se guardan en la base de datos solamente si tienen actividad social, para no sobrecargar la API Rest.

3.2.2. Definición de componentes

En Android Studio las tarjetas que definen los torrents están hechas usando el componente CardView. Estos se reciclan en un RecyclerView con scroll para ahorrar recursos. Hay distintos Intents para cada una de las 8 pantallas. Para los inputs de la pantalla de cuenta o de la barra de búsqueda se usan TextInput EditText, los botones están hechos con Button simplemente. Para la foto de perfil, como es un elemento circular, usamos un .xml personalizado que lo forme.

En React, reutilizamos los elementos de las tarjetas y los botones. Para las tarjetas implementamos [React Native CardViews](#) y lo utilizamos en conjunto con [RecyclerView](#). Utilizamos ScrollView en vertical, ahí es donde utilizamos el RecyclerView.

Para los símbolos de las tarjetas utilizamos FontAwesome. Los botones son TouchableOpacity. Los inputs de los formularios de registro, la cuenta o la barra de búsqueda son TextInput.

3.3. Chat en directo

Nuestra visión no solamente incluye la posibilidad de poder descargar torrents de manera segura y libre sino también poder comunicarse con el resto de usuarios. Por ello implementamos el sistema de comentarios, pero eso se quedaba corto ante lo que nosotros queríamos añadir. Ante tal problema, también

implementamos un sistema de chats en directo de un usuario a otro, a través del cual se pueden llevar a cabo conversaciones de manera segura y privada.

3.3.1. Backend

En la superficie, un chat funciona igual que cualquier otro componente que se ha mencionado anteriormente en cuanto a la base de datos. Está constituido por dos modelos. El modelo del mensaje y el de cada habitación.

El modelo de la habitación utiliza dos ForeignKeys que apuntan a los usuarios que participan en ella, unidas por un UniqueTogether que evita que se puedan crear varias habitaciones entre usuarios y manteniendo el orden. También, como muchos de los componentes del back, basan su seguridad en JWTAuthentication. Así, solamente los miembros de la habitación son capaces de acceder a la propia habitación, manteniendo la seguridad en todo momento.

Los mensajes funcionan de manera distinta que el resto de los modelos. Para poder mandar y recibir mensajes en vivo, la base de datos necesita poder recibir y enviar información por WebSocket. Para poder trabajar con WebSockets y HTTP al mismo tiempo, se utiliza la librería llamada “Daphne” junta con “Channels”. Estas dos librerías en conjunto permiten que la base de datos detecte si una llamada es de tipo WebSocket o HTTP. Al recibir un mensaje WebSocket, este es redirigido a un archivo “consumers.py” que se encarga de procesar la información y guardar el mensaje en la base de datos de manera simple para después poder cargarlo al abrir el chat de nuevo.

3.3.2. Frontend

El chat en el frontend es muy sencillo. Utiliza un recyclerview para mostrar los mensajes. Las dos principales diferencias es que la petición que hace a la base de datos es a través del enlace WebSocket y que el RecyclerView tiene dos holders distintos; uno para mensajes mandados y otro para mensajes recibidos.

4. Estilo de la aplicación

4.1. Paleta de colores

#48457C (Azul pizarra): Este color, además de ser principal, es utilizado porque representa ese “lado oscuro” de las aplicaciones y páginas piratas, dándole un aire a esas páginas. Este color se utiliza para los headers, haciendo que resalten.

Siguiendo la teoría del color, complementamos con otros colores que combinen bien y que no sean malos visualmente.

#FF8400 (Naranja brillante): Este color también es principal, hace un cambio un poco brusco pero para nada desagradable, pues estos colores se utilizan para botones principalmente o también para fondos de entradas de texto. Es un color complementario del azul pizarra. Este color busca ser ese cambio de imagen de la aplicación.

#EEAD4D (Naranja claro): Esta es una variación del naranja brillante, siendo un poco más tranquila, utilizándose para los comentarios que los usuarios hagan.

#E0D222 (Amarillo mostaza): Este color se utiliza únicamente para el icono de favorito, es para un detalle pequeño, pero que sirve un poco para resaltar y mostrar diferencia de todo lo demás.

#000000 (Negro): Utilizado para bordes, sombras, texto, iconos, aportando contraste y profundidad a la aplicación.

#FFFFFF (Blanco): Utilizado para fondos, texto, iconos. Muestra claridad contrastando el header y acompañando el naranja y resaltando este mismo con el blanco.

4.2. Fuente

La principal fuente utilizada en la aplicación es Zud Juice, una de las tres fuentes usadas en las “scanlations” de los mangas². Esto va a la par con algunos elementos visuales que se mencionan en el siguiente apartado para crear una experiencia más cercana a nuestro usuario promedio.

4.3. Elementos visuales

² [AnonBlack's Typesetting Guide](#)

Las tarjetas de los torrents son cuadradas y llenas de información como si fueran paneles de manga. Las casillas de los inputs tienen bordes ligeramente redondeados para parecer más ligeros a la vista. Los headers presentan un fondo estilizado con escenas de anime o de manga, dependiendo de la pantalla en la que se encuentre el usuario. Los iconos son de Material 3 en el caso de Android y de FontAwesome en el caso de React. Encima de la barra de búsqueda hay un pequeño detalle , una ilustración del Detective Conan con su lupa elevada, como si estuviera buscando. En la pantalla de perfil, el espacio donde se puede escribir la descripción está representado por una burbuja de texto, como si el usuario estuviese escribiendo un diálogo él mismo

5. Memoria del desarrollo

En esta memoria se presenta una evolución del proyecto, destacando aquellos detalles que han sido complicados de implementar, o que hayan tenido alguna parte distinta al resto.

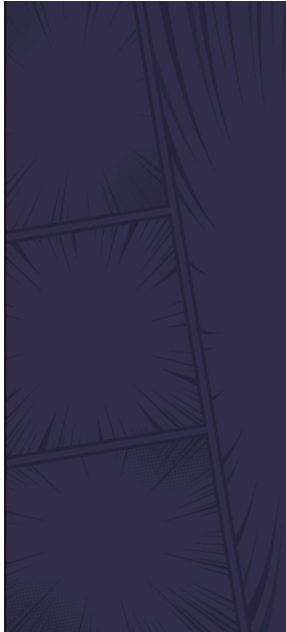
5.1. Fase de diseño

- OpenAni (v1):

- Color: Lo primero en lo que pensamos al tener la idea de la aplicación fue en el color. Fuimos variando las opciones y decidimos utilizar tonalidades de azul, pero en una versión oscura; nos habíamos inspirado en el color de la página [“HiAnime”](#).
- Diseño: Avanzamos con el diseño desde Figma. Lo que más se nos complicó fue crear el diseño principal para la tarjeta de los torrents. Esta tarjeta es el elemento más importante de la aplicación, ya que es lo que el usuario utilizará con mayor frecuencia. Teníamos en cuenta implementar en ellas las opciones de favoritos, likes y dislikes; sin embargo, solo se mantuvieron los favoritos, ya que no encontrábamos la forma de aplicar los likes y dislikes.
- Resumen: En general, la aplicación estaba quedando como cualquier otra de anime: colores oscuros, no resaltaba en casi nada y los botones apenas se notaban. Había muchos puntos a tener en cuenta mientras hacíamos el diseño; las pantallas eran muy básicas y tenían pocos elementos llamativos. Por lo tanto, nos dimos a la tarea de pensar en cómo mejorarla mediante la implementación de nuevos elementos y diseños.

En este punto aún no habíamos pensado en un logo. Teníamos ideas de lo que queríamos, pero no lo veíamos claro, así que solo usábamos texto como placeholder en el lugar donde iría este.

En este punto aún no habíamos pensado en un logo, teníamos ideas de lo que queríamos, pero aún así no lo teníamos muy claro, solo usábamos texto como placeholder de donde iría este.



< Login

Logo :)

USERNAME
Alex

EMAIL
Alex@gmail.com

PASSWORD
.....

LOG IN

DON'T HAVE AN ACCOUNT? FORGOT PASSWORD?

< Register

USERNAME
Alex

EMAIL
Alex@gmail.com

PASSWORD
.....

CONFIRM PASSWORD
.....

SIGN UP

ALREADY HAVE AN ACCOUNT?

ALEX

Anime

- MUSIC VIDEO
- ENGLISH
- NON ENGLISH
- ORIGINAL

Manga

- ENGLISH
- NON ENGLISH
- ORIGINAL

< Title ☆

SPECIFICATIONS

CATEGORY: ANIME - ENGLISH-TRANSLATED
SIZE: 271.6 MB
LAST UPDATE: 2026-02-04 16:11

DOWNLOAD

COMMENTS

ALEX
Top anime 🍌
0 0 0 04/02/2026

ALEX
Top anime 🍌

≡ Anime
Music Video

SEARCH ANIME...

TITLE	19.68	2026-01-29	125	5	☆
DOWNLOAD					
TITLE	19.68	2026-01-29	125	5	☆
DOWNLOAD					
TITLE	19.68	2026-01-29	125	5	☆
DOWNLOAD					
TITLE	19.68	2026-01-29	125	5	☆
DOWNLOAD					

- OpenAni (final):

Después de estar escuchando recomendaciones, viendo más referencias, y hablando entre nosotros, cambiamos varias cosas de la interfaz que se pensó al inicio, entre ellas: Colores, diseños, ideas e implementaciones para hacerlo más estilo de manga.

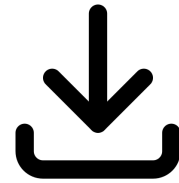
- Color: Lo primero que cambiamos fue el color. Hemos variado y cambiamos algunas cosas que eran tonalidades del azul a colores blancos y naranjas, variando estas para que fueran claras y similares a los de un manga, además de esta manera la tipografía elegida queda bien junto a los colores cambiados. Para ello, se optó por seguir la teoría del color y utilizar el naranja, que es el complementario del azul (nuestro color principal). El uso de este tono y sus variaciones, tanto claras como oscuras, le aportan una identidad única a la aplicación.

- Diseño: También hemos cambiado, mejorado y añadido más pantallas a la aplicación, como la de carga (splash screen). Para evitar un fondo plano con relieve de cuadros de manga, decidimos transformarla: ahora muestra una recopilación de distintas escenas emblemáticas de mangas como fondo y, posteriormente, introduce el logo y el nombre de la aplicación en el centro.

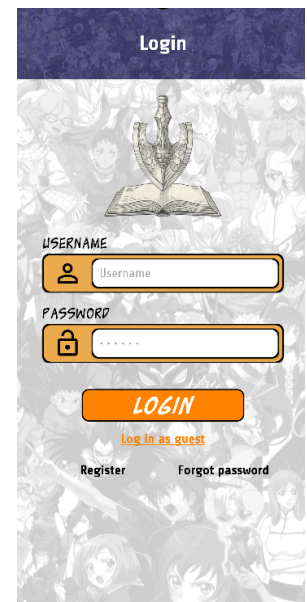
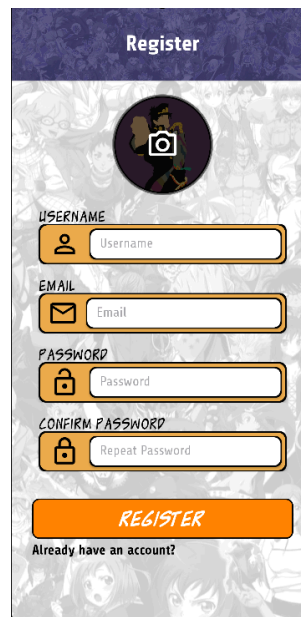
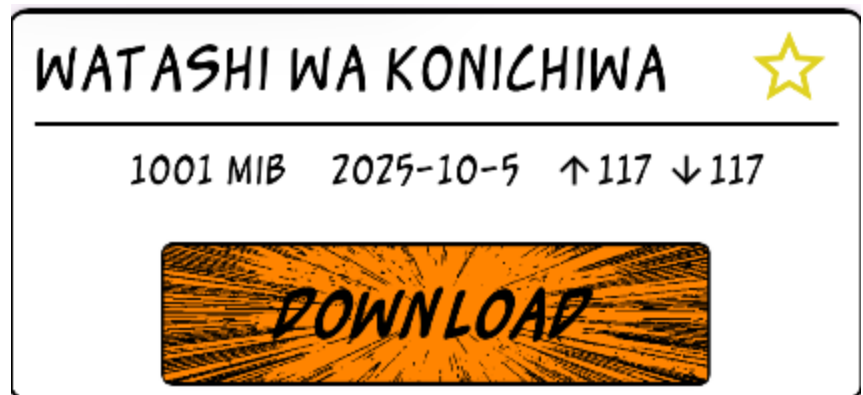
El logo de la aplicación está basado en dos animes, la flecha representa una flecha de stand, de JoJo 's Bizarre Adventure y el libro que tiene debajo es la Death Note del anime del mismo nombre.



Todo esto para hacer referencia también para hacer referencia a el botón de descargar intercambiando la flecha por una realista y lo de abajo por un manga.



Algo que nos tomó un buen tiempo fue el diseño de las tarjetas de los torrents, pues esta pasó por distintos diseños y colores, además de tener en cuenta los descartes se han hecho mediante se avanzaba con la aplicación.



5.2. Backend

- API:

El proceso de selección de la API en la que se basaría OpenAni fue relativamente largo. Mirábamos sitios web que ofrecen APIs públicas gratuitas pero no encontrábamos nada que nos atrajese la atención. En [Public Apis \(ArindamOSS en Github\)](#) se encuentran muchísimas APIs gratuitas y empezamos mirando por ahí hasta que un día a uno de nosotros le surgió una idea. Buscó en internet para ver si la página que usaba para descargar anime tenía una API y efectivamente. Encontró un proyecto en Github ([Vivek Kolhe - NyaaAPI](#)) que tenía la funcionalidad de ser una API para la página de anime.

- JWTAuthentication:

Haber hecho la base en Django es muy útil, ya que automatiza mucho el trabajo. Es un backend muy complejo y completo que facilita la creación de relaciones entre tablas de manera muy rápida a través de ForeignKeys y también automatiza la seguridad y la autenticación de los usuarios con el uso de los tokens. Un token es una cadena de caracteres aleatorios ligadas a un usuario en el momento en el que se inicia sesión. Este token se utiliza cuando se quiere ver el perfil de alguien, cuando se mandan mensajes, cuando se dejan comentarios, etc. para autenticar al usuario y saber quién es verdaderamente.

```
class UserView(APIView): 2 usages  👤 StoianAlejandro
    permission_classes = [IsAuthenticated]

    def patch(self, request):  👤 StoianAlejandro
        user = request.user
        nombre_viejo = user.username
```

Aquí por ejemplo, solamente dejaría modificar el perfil y verlo a aquella persona que se autentique y cuyos datos de usuario coincidan con las del usuario ligado al token. Esto se hace mandando

a las peticiones un parámetro en el header llamado “Authorization” seguido de “Bearer {token}”.

```
public void guardarCambios(){
    SharedPreferences prefs = PreferenceManager.getDefaultSharedPreferences( context, this);
    String token = "Bearer " + prefs.getString( key, "token", defValue: "");

    String nuevoNombre = nombreUsuario.getText().toString().trim();
    String nuevoCorreo = correo.getText().toString().trim();
    String nuevaContraseña = contraseña.getText().toString().trim();
    String nuevaDescripcion = descripcion.getText().toString().trim();
    String nuevaImagen = prefs.getString( key, "imagen", defValue: "");

    SharedPreferences.Editor editor = prefs.edit();
    editor.putString("nombre", nuevoNombre);
    editor.putString("email", nuevoCorreo);
    editor.putString("descripcion", nuevaDescripcion);
    editor.putString("imagen", nuevaImagen);

    UserUpdateModel userUpdateModel = new UserUpdateModel(nuevoNombre, nuevoCorreo, nuevaContraseña, nuevaImagen, nuevaDescripcion);
    DjangoClient.getUserAPI_Interface().updateProfile(token, userUpdateModel).enqueue(new retrofit2.Callback<UserDataModel>() {

        @Override
        public void onResponse(Call<UserDataModel> call, Response<UserDataModel> response) {
            editor.apply();
            if (response.isSuccessful()) {
                // ...
            }
        }
    });
}
```

Tardamos en entender cómo funciona el tema de los tokens pero una vez que lo entendimos fue increíble, porque agilizó mucho las peticiones HTTP y las verificaciones que se debían hacer en Django. Sin los tokens tendríamos que haber creado la lógica de búsqueda y encuentro del usuario en la base de datos nosotros mismos.

- Modelos: (importación de nombres / init.py)

Los modelos le indica a la base de datos la estructura que tendrán las tablas que se guarden; los campos que tendrán y el tipo de información que van a guardar.

```
from django.db import models
from Messaging.models import RoomModel
from Users.models import UserModel

class MessageModel(models.Model):
    room = models.ForeignKey(RoomModel, on_delete=models.CASCADE, related_name='messages')
    sender = models.ForeignKey(UserModel, on_delete=models.CASCADE,)
    text = models.TextField()
    timestamp = models.DateTimeField(auto_now_add=True)

    class Meta:
        ordering = ['timestamp']
```

Este es el modelo de mensajes que claramente guarda la sala a la que pertenece, el emisor, el texto y el momento cuando se mandó.

Tuvimos un gran problema relacionado a los modelos muy pronto en el desarrollo de OpenAni. En Django la importación de modelos entre clases se hace usando archivos .init que permiten acortar el

texto que hay que poner en los imports para que sea más fácil. Nuestro problema era que los habíamos puesto en el lugar equivocado y las direcciones relativas de dentro de los inits se estaban volviendo cada vez más complejas y difíciles de entender. Esto se arregló simplemente moviendo una carpeta más cerca de la carpeta base del proyecto.

Así es un .init correcto:

```
from .favorito_model import FavoritoModel
from .user_model import UserModel
from .reset_password_model import ResetPassword
```

Permite hacer los imports más fáciles:

```
from django.db import models
from Messaging.models import RoomModel
from Users.models import UserModel
```

- Serializers:

Los serializers son los encargados de traducir lo que se envía desde Android a Django y viceversa. Permiten la transmisión de JSONs entre ambos. También aquí es donde se elabora la lógica de verificaciones y otros.

```
class RegisterSerializer(serializers.ModelSerializer):
    username = serializers.CharField(required=True)
    email = serializers.EmailField(required=True)
    imagen = serializers.CharField(required=True)
    password = serializers.CharField(write_only=True)
    confirm_password = serializers.CharField(write_only=True)

    class Meta:
        model = UserModel
        fields = ('username', 'email', 'imagen',
                  'password', 'confirm_password')

    def validate_email(self, value):
        user = UserModel.objects.filter(email=value)

        if user:
            raise serializers.ValidationError("Email already registered")

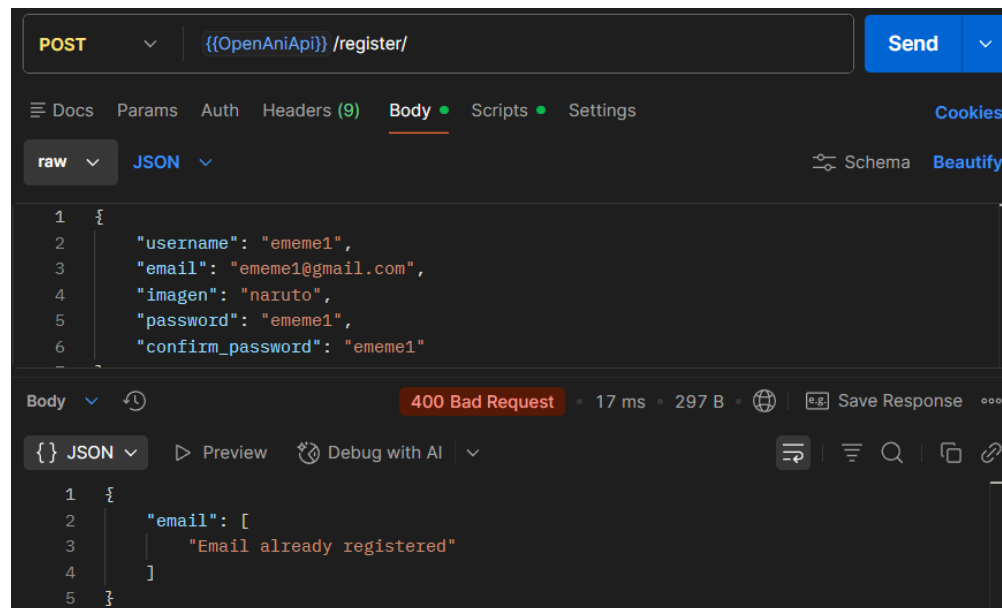
        user = UserModel.objects.filter(username=value)
        if user:
            raise serializers.ValidationError("Username already in use")
        return value

    def validate_password(self, value):
        tiene_numero = any(letra.isdigit() for letra in value)
        if not tiene_numero:
            raise serializers.ValidationError("Password must contain at least one number")
        return value

    def validate(self, attrs):
        if attrs['password'] != attrs['confirm_password']:
            raise serializers.ValidationError("Passwords don't match")
        return attrs
```

En el RegisterSerializer por ejemplo, se validan los distintos campos que se envían desde Android y, si no se consigue, se devuelven

errores específicos en los campos que hayan salido mal. Así, el control de errores también está automatizado. Si por ejemplo el “username” está mal, Django no devuelve un error y ya, sino que devuelve un JSON con el error en su campo.



Como se puede ver, el error no solamente salta y ya, sino que indica cuál es el error específico. Esto se puede recoger después en Android y mostrarlo en la casilla correspondiente.

- Migraciones, caché y control de versiones:

Durante el desarrollo tuvimos varios problemas relacionados a git y control de versiones. Pycharm suele crear archivos de caché que se deben de excluir en un .gitignore pero nosotros no hicimos eso porque se nos olvidó. A eso se sumó el hecho de que Django necesita hacer migraciones y tiene un control de versiones interno de estas migraciones. Los dos se fueron sustituyendo y completando al hacer push y pull hasta que no se podía administrar más. En ese punto, lo que hicimos fue reconstruir el proyecto entero de la base de datos para así eliminar el caché y también las migraciones. Creamos un .gitignore y cada vez que hacíamos un pull request borramos las migraciones locales para que no diesen conflictos. Gracias a ello hay un momento en el árbol de commits que no tiene ningún commit sustancial, sino solamente pruebas para ver si habíamos arreglado el problema o no.

- Consumers:

Consumers es similar a una vista normal de Django. Pero en lugar de hacer peticiones y luego detenerse, es decir, ser síncrona, este se mantiene escuchando constantemente las peticiones y reacciona

de inmediato cuando recibe información (asíncrona). Consumers además de interactuar con los modelos de usuario, room y message. También necesita un routing. Routing es la equivalente a las urls de Django. No se coloca la ruta ahí porque Django no permite el enrutamiento “ws” que se utiliza para el chat.

```
async def connect(self) -> Coroutine[Any, Any, None] :  
    self.group_id = self.scope['url_route']['kwargs']['group_id']  
    self.room_group_name = f'chat_{self.group_id}'  
    await self.channel_layer.group_add(self.room_group_name, self.channel_name)  
    await self.accept()
```

Connect se ejecuta automáticamente cuando se hacen llamadas al chat. Desde el routing se captura el id de la sala (room) y se guarda en “self.group_id”.

En “self.room_group_name” se define un nombre único para Redis. Redis es una base de datos que guarda datos en la memoria RAM, en lugar del disco duro, haciendo que esta sea muy eficaz y reaccione al instante.

Luego conecta un usuario específico actual al grupo de Redis. Y al final acepta la conexión para poder iniciar la conversación.

```
async def disconnect(self, close_code) -> Coroutine[Any, Any, None] :  
    await self.channel_layer.group_discard(self.room_group_name, self.channel_name)
```

Este se ejecuta cuando el usuario sale de la pantalla de chat, se desconecta del Wi-Fi o cierra la app directamente. Entonces desconecta al usuario del grupo Redis para que no siga intentando recibir mensajes en todo momento.

```

async def receive(self, text_data) -> Coroutine[Any, Any, None] :
    data = json.loads(text_data)
    message_text = data.get('message')
    username = data.get('sender_username')

    msg_obj = await self.save_message(username, self.group_id, message_text)

    if msg_obj:
        await self.channel_layer.group_send(
            self.room_group_name,
            message: {
                'type': 'chat_message',
                'message': msg_obj.text,
                'sender_username': msg_obj.sender.username,
                'timestamp': msg_obj.timestamp.strftime("%Y-%m-%d %H:%M:%S")
            }
        )

```

Cuando un usuario pulsa en botón “enviar”, se envían datos que consumers necesita recibir.

Primero transforma el texto recibido en diccionario, luego registra el mensaje para guardarlo en la base de datos de manera segura y asíncrona.

Si el mensaje se guardó bien hace que se envíe a la base de datos de Redis, donde está el otro usuario también, haciendo que este reciba también el mensaje enviado.

```

async def chat_message(self, event) -> Coroutine[Any, Any, None] :
    await self.send(text_data=json.dumps(event))

```

Esto se utiliza en el método “receive” desde Django. Este activa el consumer de cada usuario que esté conectado al chat.

Envía: el mensaje, el remitente y la hora a través del WebSocket a la pantalla de la interfaz en Android.

```
@database_sync_to_async 1 usage
def save_message(self, username, room_id, text) -> MessageModel | None :
    try:
        user = UserModel.objects.get(username=username)
        room = RoomModel.objects.get(id=room_id)
        return MessageModel.objects.create(room=room, sender=user, text=text)
    except Exception as e:
        print(f"Error guardando mensaje: {e}")
        return None
```

Esta función también se utiliza en “recieve”.

Es una herramienta que se comunica directamente con Django.

Teniendo en cuenta que Django siempre se mantiene síncrono habría problemas en los chats, se bloquearían los hilos mientras busca o guarda datos. Congelaría todos los chats por unos milisegundos hasta que termine de hacer lo que necesite.

Para evitar eso se coloca “@database_sync_to_async”, para que pueda hacer sus acciones sin que rompan el rendimiento en tiempo real del servidor.

- WebSockets:

Un Websocket es un canal de comunicación bidireccional y full-duplex. Esto significa que permite la comunicación entre dos partes al mismo tiempo y en tiempo real. Los websockets también están diseñados para aceptar no solamente conexiones websocket sino también HTTP. Esto permite que un mismo servidor pueda proveer ambos tipos de conexiones, siempre y cuando el servidor se abra en una URI del tipo ws://{url}/. El prefijo ws le indica al servidor que es una conexión WebSocket. También evitan problemas con el cortafuegos. Los WebSockets trabajan sobre un solo puerto TCP lo cual no haría saltar el cortafuegos, porque no está sobrecargando los puertos disponibles que tenga el servidor. Se pueden tener varias conexiones WebSocket abiertas en un mismo puerto TCP, lo cual sí implica una leve sobrecarga del protocolo pero es indiferente.

Nosotros utilizamos los WebSockets para el chat en directo. Al permitir una comunicación en tiempo real entre servidor y cliente, se puede hacer una cadena de información para convertirla en cliente -

cliente: cliente (emisor) -> servidor (intermediario) -> cliente (receptor).

Siguiendo la guía de [GeeksForGeeks \(Learn to use Websockets with Django\)](#) fuimos capaces de crear este tipo de conexión en nuestra base de datos.

En Django, el servidor utiliza por defecto el archivo llamado wsgi.py para levantarse. Para comenzar con el servidor del tipo websocket tuvimos que indicarle en su settings.py que el archivo que va a utilizar es [asgi.py](#).

```
ASGI_APPLICATION = 'OpenAni_DB.asgi.application'
```

```
import ...
## import OpenAni_DB.routing
##### HAY QUE CREAR MODELO CHATCONSUMER (MIRAR LA PAGINA GUIA Y EL GEMINI) listo cruck
##### HAY QUE CREAR EL ARCHIVO ROUTING.PY DESPUÉS listo cruck

os.environ.setdefault(key='DJANGO_SETTINGS_MODULE', value='OpenAni_DB.settings')

django_asgi_app = get_asgi_application()

application = ProtocolTypeRouter({
    "http": django_asgi_app,

    "websocket": AuthMiddlewareStack(
        URLRouter(
            routing.websocket_urlpatterns
        )
    ),
})
```

La imagen anterior muestra el archivo asgi.py.

```
# chat/routing.py
import ...

websocket_urlpatterns = [
    # Usamos re_path para capturar el group_id de la URL
    # Esta ruta coincide con la que pongas en Android Studio
    # 'ws/chat/(?P<group_id>w+)/$' -> es un capturador. Lo que está entre <> es el nombre de la variable que se usa en consumers.
    # Hace que se pueda llamar en consumers como: self.group_id = self.scope['url_route']['kwargs']['group_id']
    # w+ -> permita valores alfanuméricos con más de 1 dígito.
    re_path(route='ws/chat/(?P<group_id>w+)/$', consumers.ChatConsumer.as_asgi()),
]
```

Como indicamos anteriormente, el servidor debe funcionar bajo una url de tipo ws://. El archivo routing.py le indica el enlace que debe usar.

- Peticiones HTTP desde Android:

Las peticiones CRUD en Android no son demasiado difíciles gracias a una librería llamada [Retrofit](#) que permite hacer magia a través de unas etiquetas que tiene hardcodeadas para hacer las peticiones.

```

public interface ChatInterface {

    1 usage
    @GET("messages/")
    Call<MessageModel> getMessage(@Header("Authorization") String token, @Query("room_id") String group_id);

    2 usages
    @GET("rooms/")
    Call<RoomModel> getRooms(@Header("Authorization") String token);

    1 usage
    @POST("rooms/")
    Call<RoomModel.RoomDetail> createRoom(
        @Header("Authorization") String token,
        @Body Map<String, String> body
    );
}

```

Utiliza @GET, @POST, @PATCH y @DELETE para indicar el tipo de solicitud.

Encontramos varias dificultades. La primera fue cómo utilizar el token en Retrofit. Por suerte, también tiene una etiqueta para eso, @Header, que permite añadir encabezados a las peticiones.

Otra dificultad fue cómo hacer patches y peticiones con queries:

```

1 usage
@GET("users/")
Call<List<UserDataModel.UserData>> getProfiles(@Header("Authorization") String token, @Query("username") String username);

1 usage
@PATCH("users/")
Call<UserDataModel> updateProfile(@Header("Authorization") String token, @Body UserUpdateModel userUpdateModel);

```

@Query permite añadir parámetros extra a las búsquedas para mandar enlaces del tipo: users/{query}=x.

@Patch sirve para modificar parcialmente una entrada de la base de datos. Este necesita una etiqueta @Body cuyo objeto debe incluir lo que se vaya a cambiar. Utiliza el UserUpdateModel, que puede tener Strings vacíos que no se tienen en cuenta al hacer el Patch.

```

public class UserUpdateModel {
    @SerializedName("username")
    public String username;

    @SerializedName("email")
    public String email;

    @SerializedName("password")
    public String password;

    1 usage
    @SerializedName("imagen")
    public String imagen;

    1 usage
    @SerializedName("descripcion")
    public String descripcion;

    2 usages
    public UserUpdateModel(String username, String email, String password, String imagen, String descripcion) {
        this.username = username;
        this.email = email;
        this.password = password;
        this.imagen = imagen;
        this.descripcion = descripcion;
    }
}

```

- Control de Errores:
Login:

```

private void mostrarErrorVacio(){
    new AlertDialog.Builder( context: this)
        .setTitle("Ingresa tus datos")
        .setMessage("Faltan datos por rellenar, por favor ingrézalos.")
        .setIcon(R.drawable.alert)
        .setPositiveButton( text: "Aceptar", listener: null)
        .show();
}

```

Este control de error sirve para darle más feedback al usuario, cuando no haya ingresado uno de sus datos en una casilla. Esto es mostrado en un AlertDialog

Register:

```

public boolean emptyVerify(EditText editText){
    String text = editText.getText().toString();
    if(text.isEmpty()){
        Toast.makeText( context: this, text: "Error, falta(n) rellenar campos ", Toast.LENGTH_SHORT).show();
        editText.setError("Por favor, introduzca su usuario");
        return false;
    } else {
        return true;
    }
}

1 usage
public boolean emailVerify(EditText emailEditText){
    String email = emailEditText.getText().toString();
    String EMAIL_REGEX = "[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\\.[A-Za-z]{2,}$";
    if(email.isEmpty()){
        emailEditText.setError("Por favor, introduzca su email");
        return false;
    }
    if(!Pattern.matches(EMAIL_REGEX, email)) {
        emailEditText.setError("Email no válido");
        return false;
    }
    return true;
}

```

```

public boolean comprobarContraseñas(EditText contral, EditText contra2){
    String firstContra = contral.getText().toString();
    String secondContra = contra2.getText().toString();

    if (firstContra.isEmpty() && secondContra.isEmpty()) {
        Toast.makeText( context: this, text: "Este campo no debe estar vacío", Toast.LENGTH_SHORT).show();
        contral.setError("Por favor, introduzca su contraseña");
        contra2.setError("Por favor, introduzca su contraseña nuevamente");
        return false;
    }
    if (!firstContra.matches( regex: "^(?=.*[0-9])(?=.*[a-z]).{6,}$")) {
        Toast.makeText( context: this, text: "La contraseña debe tener al menos 6 caracteres y un número", Toast.LENGTH_LONG).show();
        contral.setError("Las contraseñas deben tener, Min 1 número y 6 caracteres");
        contra2.setError("Las contraseñas deben tener, Min 1 número y 6 caracteres");
        return false;
    }
    if (!firstContra.equals(secondContra)) {
        Toast.makeText( context: this, text: "Las contraseñas no coinciden. Por favor, vuelve a intentarlo", Toast.LENGTH_SHORT).show();
        contral.setError("Las contraseñas deben coincidir");
        contra2.setError("Las contraseñas deben coincidir");
        return false;
    }
    return true;
}

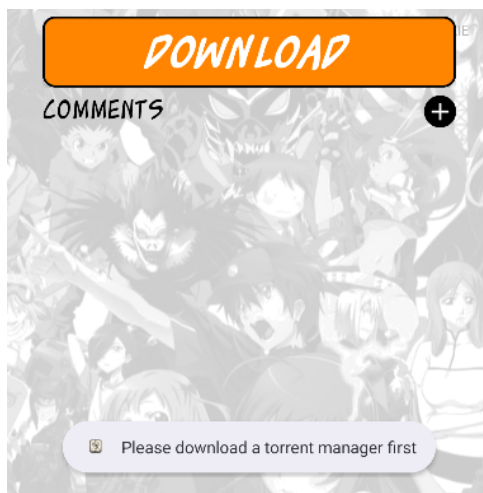
```

En el método de emptyVerify() se verifica que ningún campo esté vacío, por eso se le pasa un EditText. Si llega a estar vacío se le muestra al usuario un toast con un mensaje que le comunica la falta de campos.

Luego en emailVerify() se utiliza un REGEX para admitir correos que tengan los caracteres estándares. Luego verifica si está vacío y también si no coincide con el REGEX salta un error que el email no es válido.

Y por último en comprobarContrasenias() se comprueba que si una contraseña u otra están vacías se manda un toast que ese campo falta rellenar. Luego se verifica con un REGEX que si las contraseñas no cumplen con 1 número y 6 caracteres como mínimo se manda otro toast. Finalmente se comprueba que ambas contraseñas coinciden.

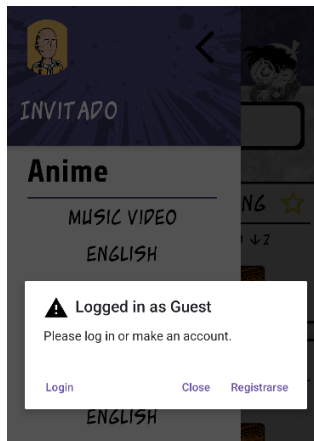
Download torrent:



```
boton_descargar.setOnClickListener(new View.OnClickListener() {  
    @Override  
    public void onClick(View v) {  
        Intent intent = new Intent(Intent.ACTION_VIEW, Uri.parse(enlace.toString()));  
        if (intent.resolveActivity(getPackageManager()) != null) {  
            startActivity(intent);  
        } else {  
            Toast.makeText(context, "Please download a torrent manager first", Toast.LENGTH_LONG).show();  
        }  
    }  
});
```

Esta función hace que cuando se haga clic en el botón DOWNLOAD de cualquier torrent que se quiera descargar si no tienes un administrador o descargador de torrents bajado, no te permitirá descargar nada y te muestra un Toast. En caso de que si lo tengas se podrá descargar con normalidad.

Invitado:



```
private void errorInvitado() {
    new AlertDialog.Builder(context: this)
        .setTitle("Logged in as Guest")
        .setMessage("Please log in or make an account.")
        .setIcon(R.drawable.alert)
        .setPositiveButton(text: "Registrarse", new DialogInterface.OnClickListener() {
            @Override
            public void onClick(DialogInterface dialog, int which) {
                Intent intent = new Intent(packageContext: MainAnime.this, RegisterView.class);
                startActivity(intent);
            }
        })
        .setNeutralButton(text: "Login", new DialogInterface.OnClickListener() {
            @Override
            public void onClick(DialogInterface dialog, int which) {
                Intent intent = new Intent(packageContext: MainAnime.this, LoginView.class);
                startActivity(intent);
            }
        })
        .setNegativeButton(text: "Close", listener: null)
        .show();
}
```

Para los usuarios que hayan ingresado a la aplicación como invitados, existen ciertos límites para ellos. Por ello mostramos un AlertDialog que es un método que llamamos cuando se hace clic en Favoritos, Foto de perfil y Chat.

```
foto_perfil.setOnClickListener(new View.OnClickListener() {
    @Override
    public void onClick(View v) {
        if (esInvitado()) {
            errorInvitado();
        } else {
            Intent intent = new Intent(packageContext: MainAnime.this, AccountView.class);
            startActivity(intent);
        }
    }
});
```

```
favorites.setOnClickListener(new View.OnClickListener() {
    @Override
    public void onClick(View v) {
        if (esInvitado()) {
            errorInvitado();
            return;
        }
        header_categoria.setText("Favoritos");
        header_subcategoria.setText("");
        fillFavorites();
        cerrar_menu_lateral(slide_out);
    }
});
```

```
chat.setOnClickListener(new View.OnClickListener() {
    @Override
    public void onClick(View v) {
        if (esInvitado()) {
            errorInvitado();
        } else {
            Intent intent = new Intent(packageContext: MainAnime.this, AllChatsActivity.class);
            startActivity(intent);
        }
    }
});
```

Aquí es donde llamamos a nuestra función errorInvitado(), esto con un if donde primero verificamos si es un invitado. Si lo es, mostramos nuestra función antes mencionada. Si no, se hará un intent para entrar a la actividad correspondiente.

5.3. Frontend

- Splash Screen:



En nuestro SplashScreen se pueden apreciar varios paneles de mangas conocidos de diferentes categorías. Para crear este splashScreen se tuvo que buscar diferentes paneles de manga y hacer que encajen todos entre sí, dando la impresión que se trata de un solo manga. Un detalle es que no fue sencillo juntar imágenes para el fondo, pues se tuvo que hacer que todos coincidieran y además buscar paneles de mangas exactos para que encajen uno junto a otro.

```

<ImageView
    android:id="@+id/splashImagenFondo"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:scaleType="centerCrop"
    tools:layout_editor_absoluteX="0dp"
    tools:layout_editor_absoluteY="0dp">

</ImageView>

<View
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:alpha="0.8"
    android:background="@color/black"
    tools:layout_editor_absoluteX="0dp"
    tools:layout_editor_absoluteY="0dp">

</View>

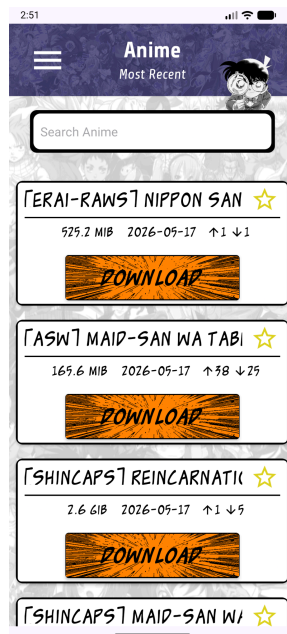
<ImageView
    android:id="@+id/splashLogo"
    android:layout_width="268dp"
    android:layout_height="268dp"
    android:src="@drawable/logo prueba"
    app:layout_constraintBottom_toBottomOf="parent"
    app:layout_constraintEnd_toEndOf="parent"
    app:layout_constraintStart_toStartOf="parent"
    app:layout_constraintBottom_toTopOf="@id/v1">

</ImageView>

```

para crear el SplashScreen tuve que usar dos ImageView, un para el fondo en sí que son todos los paneles de mangas juntos y otro para el logotipo. El logotipo va solapado sobre el fondo para dar un efecto de profundidad, esto se consiguió con un `background="@color/black"` sobre el fondo.

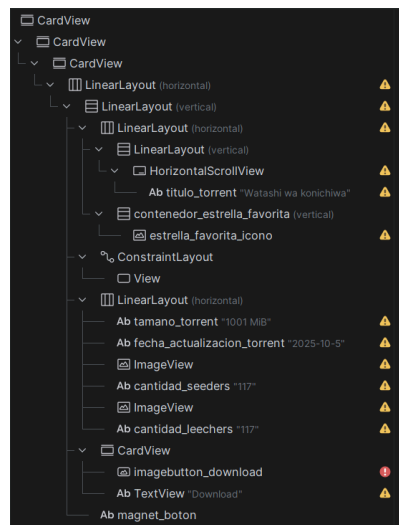
- Pantalla de Torrents:



Las tarjetas de los torrents acabaron siendo uno de los primeros desafíos al que nos enfrentamos, por el simple hecho de que ordenar los distintos elementos de dentro de la tarjeta fue complicado. Siendo una de los primeros componentes que implementamos en nuestra aplicación todavía no teníamos mucha experiencia para poder hacerlo con fluidez.



Utilizamos muchos LinearLayouts anidados para poder poner los elementos justo donde queríamos y de manera uniforme. Gracias a ello hemos podido entender mejor cómo funcionan las etiquetas de “weight” o “layout_weight”, ya que tuvieron un gran peso en el diseño de las tarjetas.



```
<FrameLayout
    android:layout_width="match_parent"
    android:id="@+id/body"
    android:layout_height="0dp"
    app:layout_constraintTop_toBottomOf="@id/header"
    app:layout_constraintBottom_toBottomOf="parent"
    android:orientation="vertical"
    android:background="@color/white"
>

<ImageView
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:src="@drawable/header_anime_fondo"
    android:scaleType="centerCrop"
    android:alpha="0.2"
```

Otro detalle que fue una novedad para nosotros fue algo muy pequeño pero que le añade personalidad a la aplicación. El dibujo de Conan que aparece en la barra de búsqueda no ha sido tan sencillo de hacer como podría parecer. Tuvimos que investigar cómo funciona la elevación y la capacidad que otorgan de poder

superponer elementos en la pantalla. Ese detalle nos dejó hacer un efecto parecido a uno 3D donde Conan se alza sobre el encabezado.

Para el fondo de las pantallas, el collage de personajes en blanco y negro, tuvimos que aprender a usar los FrameLayouts. Estos layouts permiten colocar elementos en cualquier posición, además de tener accesible la etiqueta “gravity” para ordenar esos elementos incluso mejor.

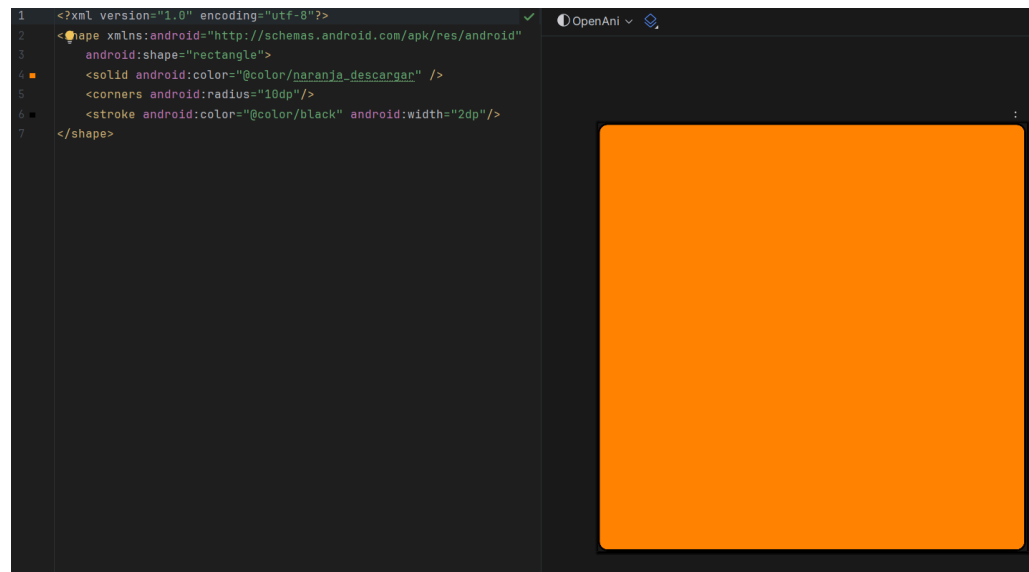
- Pantalla de Torrent Individual:

Utilizamos la misma técnica que usamos para el icono de Conan en la barra de búsqueda para poner una burbuja de texto detrás de la descripción del torrent. Reutilizamos el código del resto de pantallas para poner el fondo.

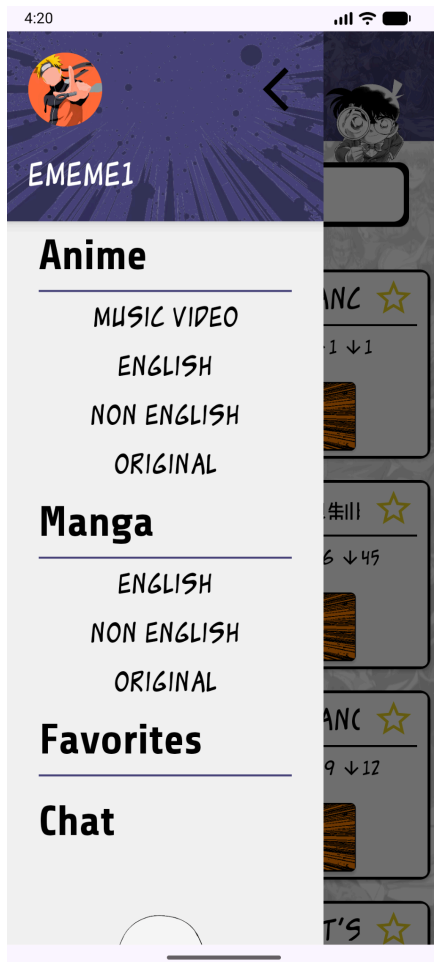


Toda la información se recoge de la API de [Nyaa.si](https://nyaa.si) a través de peticiones HTTP.

El botón de descarga se hizo con un diseño personalizado, utilizando Shape y basándose en <https://stackoverflow.com/questions/7960755/how-to-create-a-drawable-thats-square-on-the-outside-and-has-rounded-corners-in>:



- Menú Lateral:



El menú lateral fue también un gran desafío. No solamente tuvimos que ver cómo añadirlo encima del home, sino también como añadirle una sombra y animaciones. El funcionamiento interno de cuando pulsas sobre una opción del menú también es ligeramente diferente a los otros onClicks.

```
barra_lateral_icono.setOnClickListener(new View.OnClickListener() {
    @Override
    public void onClick(View v) {
        menu_lateral_visible = !menu_lateral_visible;
        sombra_menu_lateral.setVisibility(VISIBLE);
        contenedor_menu_lateral.removeAllViews();
        menu_lateral = inflater_menu_lateral.inflate(R.layout.menu_lateral, contenedor_menu_lateral, true);
        Animation slide_in = AnimationUtils.loadAnimation(getApplicationContext(), R.anim.slide_in_from_left);
        Animation slide_out = AnimationUtils.loadAnimation(getApplicationContext(), R.anim.slide_out_from_right);
        menu_lateral.setAnimation(slide_in);
        cerrar_menu_lateral = menu_lateral.findViewById(R.id.cerrar_menu_lateral);
        anime = menu_lateral.findViewById(R.id.anime);
        manga = menu_lateral.findViewById(R.id.manga);
        foto_perfil = menu_lateral.findViewById(R.id.imagen_perfil);
        nombre_usuario = menu_lateral.findViewById(R.id.nombre_usuario);
        favorites = menu_lateral.findViewById(R.id.favorites);
        chat = menu_lateral.findViewById(R.id.chat);
    }
});
```

El menú lateral se pinta dentro de un ConstraintLayout vacío e invisible que, al pulsar el icono “barra_lateral_icono”, se rellena con un inflater y se le añade una animación de entrada y otra de salida. La

sombra es simplemente un elemento que está permanentemente encima de la pantalla principal pero en estado invisible. Cuando se pinta el menú lateral simplemente se vuelve visible y crea la ilusión de ser una sombra perteneciente al menú lateral.

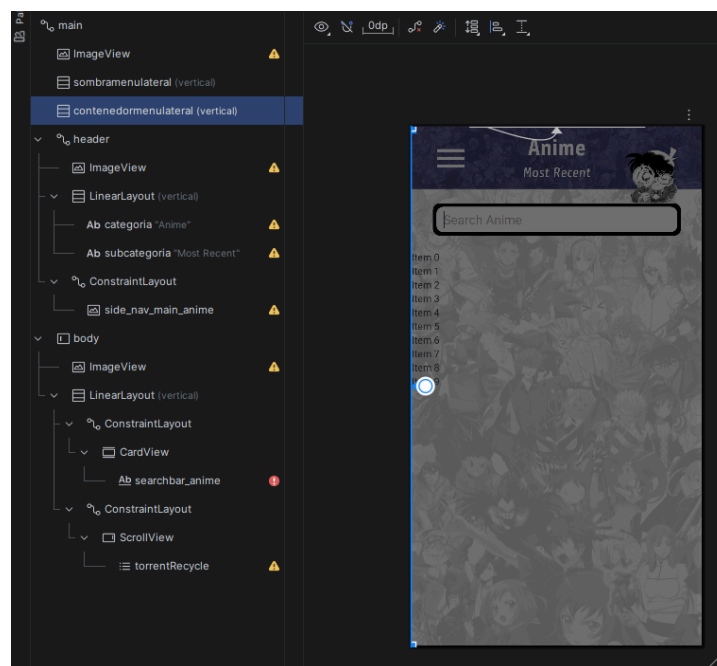
```
<set xmlns:android="http://schemas.android.com/apk/res/android"
    android:shareInterpolator="false">
    <translate android:fromXDelta="-100%" android:toXDelta="0%"
        android:fromYDelta="0%" android:toYDelta="0%"
        android:duration="200"/>
</set>
```

Esta es la animación de salida del menú lateral. Utiliza porcentajes para decidir desde

qué punto hasta qué punto se debe desplazar un elemento teniendo como referencia el punto en el que está inicialmente. -100% lo mueve fuera de la pantalla a la izquierda, por lo que al ir al 0% parece como si apareciese desde fuera hasta su posición natural.

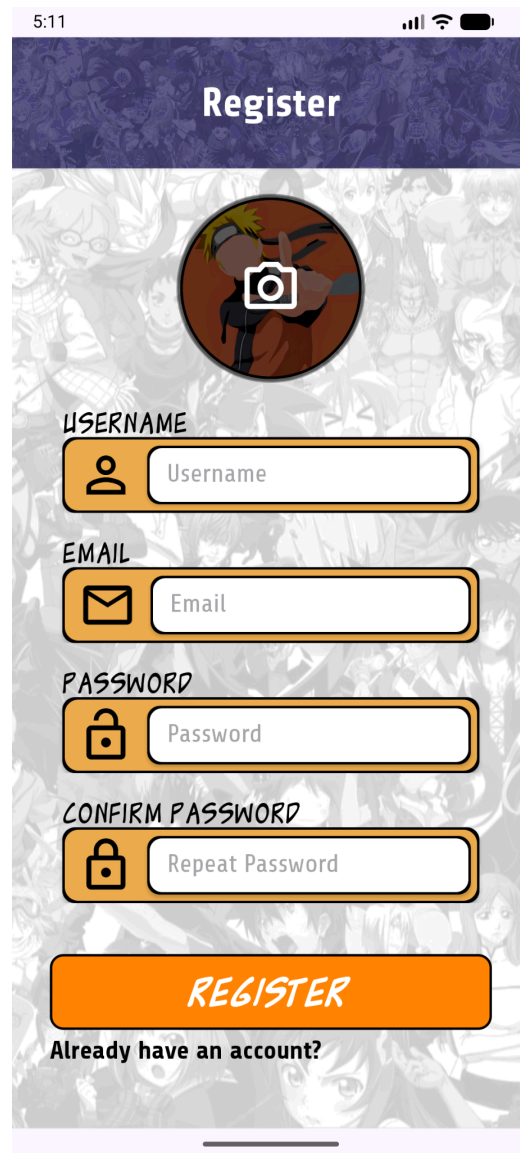
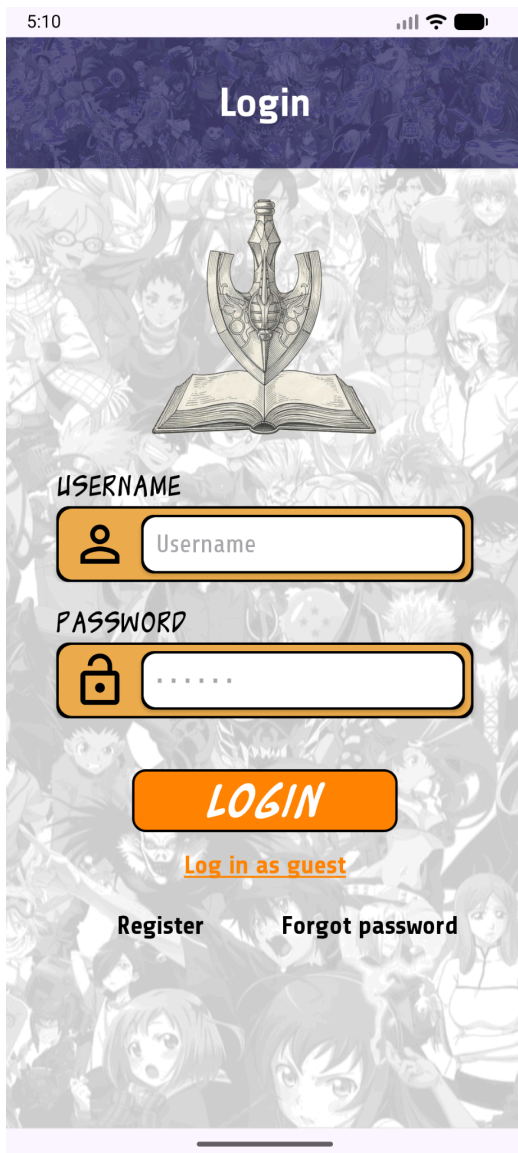
(<https://stackoverflow.com/questions/23925907/slidedown-and-slideup-layout-with-animation>)

En esta imagen se aprecia el “contenedormenulateral” vacío en el que se pinta el menú lateral y también la sombra permanente que le da un aspecto oscuro a la pantalla pero que se le añade el atributo de “INVISIBLE” en el código.

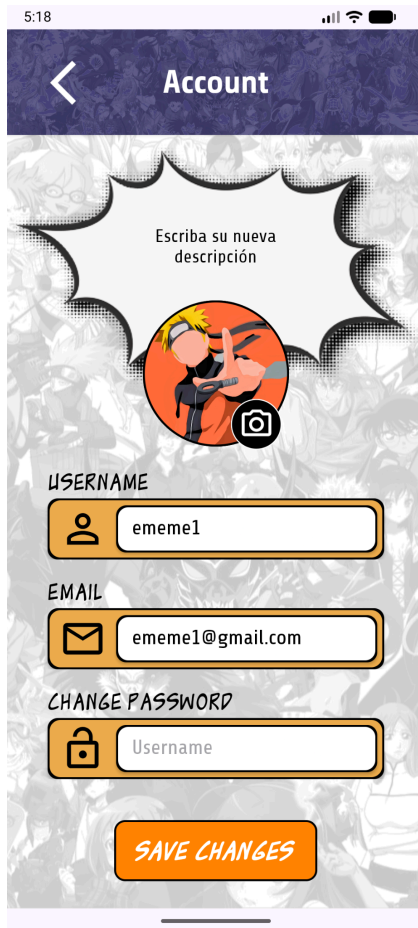


- Login y Register:

En cuanto a diseño, lo más destacable de estas pantallas es el uso de una librería externa para la imagen de perfil. Esta librería pasaría después a utilizarse en el resto de la aplicación siempre que aparecen imágenes con forma circular. La librería facilita la creación de imágenes de distintas formas para no tener que crear un Shape customizado. ([Cheonjaeung - ShapedImageView \(Github\)](#))



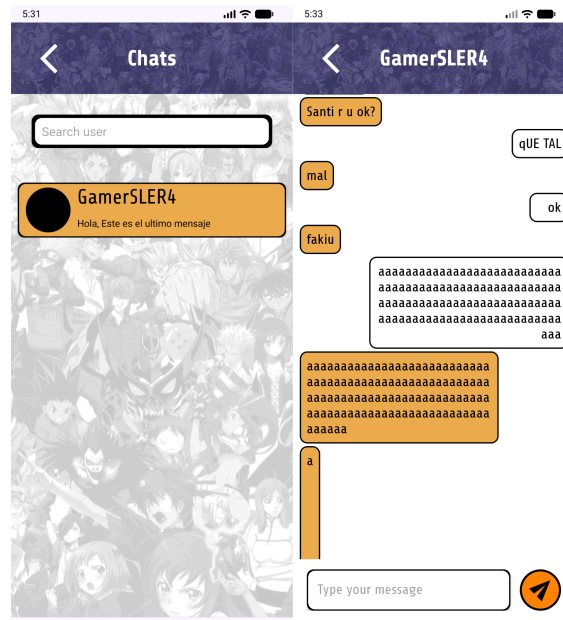
- Perfil:



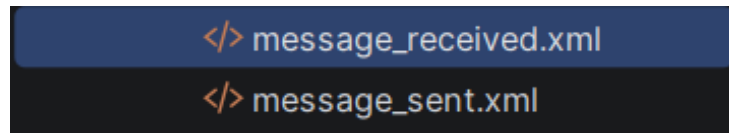
En la pantalla de perfil al pulsar el botón de Save Changes se manda una petición PATCH con datos parciales a la base de datos. Esto se explica más a fondo en el apartado Backend de la memoria. En cuanto a diseño, copiamos elementos de las pantallas de Login y Register.

Hay un ConstraintLayout invisible en la parte inferior de la pantalla, similar al que se usó para el menú lateral. Esta vez es para pintar el menú de cambiar la foto de perfil.

- Habitaciones y chat:



Las pantallas de habitaciones y chats ambas usan RecyclerViews pero de maneras distintas. La pantalla de chats es la primera vez que hemos tenido que hacer un solo RecyclerView con dos holders distintos. Para poder pintar los mensajes recibidos y enviados en el mismo RecyclerView estos se deben de ordenar de manera distinta pero sincronizada. En la parte del Backend se explica mejor.



Para ello se usan dos xmls distintos. Cada uno de los diseños de mensaje ocupa una fila entera para que en el recyclerview se pinten fila por fila y no se superpongan aunque haya problemas con el mensaje.

En la pantalla de las salas no solamente se muestra la persona con la que se está hablando sino también el último mensaje mandado en la sala.

- Menú para cambiar la foto de perfil:



En el menú para cambiar la foto de perfil de un usuario, se agregaron varias fotos predeterminadas con el mismo diseño de la aplicación. Estas se van cambiando dinámicamente si el usuario hace clic sobre una de ellas.

```

121 <LinearLayout
122     android:layout_marginBottom="15dp"
123     android:layout_width="match_parent"
124     android:layout_height="wrap_content"
125     android:orientation="horizontal"
126     android:gravity="center"
127 >
128     <LinearLayout
129         android:id="@+id/contenedorGoku"
130         android:layout_weight="1"
131         android:layout_width="0dp"
132         android:layout_height="wrap_content"
133         android:orientation="vertical"
134     >
135         <TextView
136             android:id="@+id/idImagen"
137             android:layout_width="0dp"
138             android:layout_height="0dp"
139             android:text="@string/goku_ss12"
140         >
141         </TextView>
142         <io.wuong.shapedimageview.CircleImageView
143             android:id="@+id/goku"
144             android:layout_width="100dp"
145             android:layout_height="100dp"
146             android:src="@drawable/foto_de_perfil_goku_ss12"
147             app:border_size="1dp"
148             app:border_color="@color/black"
149             android:layout_gravity="center"
150         >
151         </io.wuong.shapedimageview.CircleImageView>
152     </LinearLayout>

```

Aquí se puede ver un ejemplo de cómo se agrega una foto de perfil. Se le asigna un id al LinearLayout que va a ser el contenedor de la foto y luego un textView, para que el usuario pueda hacer clic en ella. Para agregar la foto de perfil, se utiliza el CircleImageView. Aquí fue donde se agregó la foto de la carpeta drawable.

```

botonCambiarFoto.setOnClickListener(new View.OnClickListener() {
    layoutInflater.inflate(R.layout.change_photo_of_profile, layoutInflater.inflateAccountViewReference, attachToRoot: true);
    Animation slide_in = AnimationUtils.loadAnimation(getApplicationContext(), R.anim.slide_up);
    Animation slide_out = AnimationUtils.loadAnimation(getApplicationContext(), R.anim.slide_down);
    botonCambiarFoto.setAnimation(slide_in);

    List<LinearLayout> fotosDePerfil = new ArrayList<>(Arrays.asList(
        primeraFoto = botonCambiarFoto.findViewById(R.id.contenedorKameki),
        segundaFoto = botonCambiarFoto.findViewById(R.id.contenedorJotaro),
        terceraFoto = botonCambiarFoto.findViewById(R.id.contenedorJinMoo),
        cuartaFoto = botonCambiarFoto.findViewById(R.id.contenedorGoku),
        quintaFoto = botonCambiarFoto.findViewById(R.id.contenedorGohan),
        sextaFoto = botonCambiarFoto.findViewById(R.id.contenedorNicoRobin),
        septimaFoto = botonCambiarFoto.findViewById(R.id.contenedorToga),
        octavaFoto = botonCambiarFoto.findViewById(R.id.contenedorMakashi),
        novenaFoto = botonCambiarFoto.findViewById(R.id.contenedorNaruto),
        decimaFoto = botonCambiarFoto.findViewById(R.id.contenedorPersonajeTR),
        decimoPrimeraFoto = botonCambiarFoto.findViewById(R.id.contenedorSasuke),
        decimoSegundaFoto = botonCambiarFoto.findViewById(R.id.contenedorDeJojo),
        decimoTerceraFoto = botonCambiarFoto.findViewById(R.id.contenedorMeliodas),
        decimoCuartaFoto = botonCambiarFoto.findViewById(R.id.contenedorMikasa),
        decimoQuintaFoto = botonCambiarFoto.findViewById(R.id.contenedorLuffy)
    ));

    for (LinearLayout foto: fotosDePerfil) {
        foto.setOnClickListener(new View.OnClickListener() {
            @Override
            public void onClick(View v) {
                TextView nombreFoto = v.findViewById(R.id.idImagen);
                String valor = nombreFoto.getText().toString();
                SharedPreferences sharedPreferences = PreferenceManager.getDefaultSharedPreferences(getApplicationContext());
                sharedPreferences.edit().putString("imagen", valor).apply();
                Uri uri = Uri.parse("android.resource://" + getPackageName() + "/drawable/foto_de_perfil_" + valor);
                fotoPerfil.setImageDrawable(null);
                fotoPerfil.setImageURI(uri);
                desplazarMenu(slide_out);
            }
        });
    }
});

```

```

Usage
public void desplazarMenu(Animation slide_out) {
    botonCambiarFoto.setAnimation(slide_out);
    new Handler().postDelayed(new Runnable() {
        @Override
        public void run() {
            layoutInflater.inflateAccountViewReference.removeAllViews();
        }
    }, slide_out.getDuration());
}

```

En estos métodos que se crearon en AccountView se utiliza un inflate para inflar a cada una de las fotos. Para esto primero se crea un ArrayList con cada uno de los ids, para que el inflate los vaya recorriendo en el onClick. Luego de esto en el mismo onClick se guarda la foto de perfil en el sharedPreferences y después de esto se busca con un Uri su ruta y se muestra la animación de cerrar.